EXCEPTIONS

The problem that occcers during runtime because of abnormal statements present inside the program, we call the as Exceptions.

Eg: Arithmetic - Exceptions

NullPpointerException, ClassCastException, etc..,

Eg for ArithmeticException:

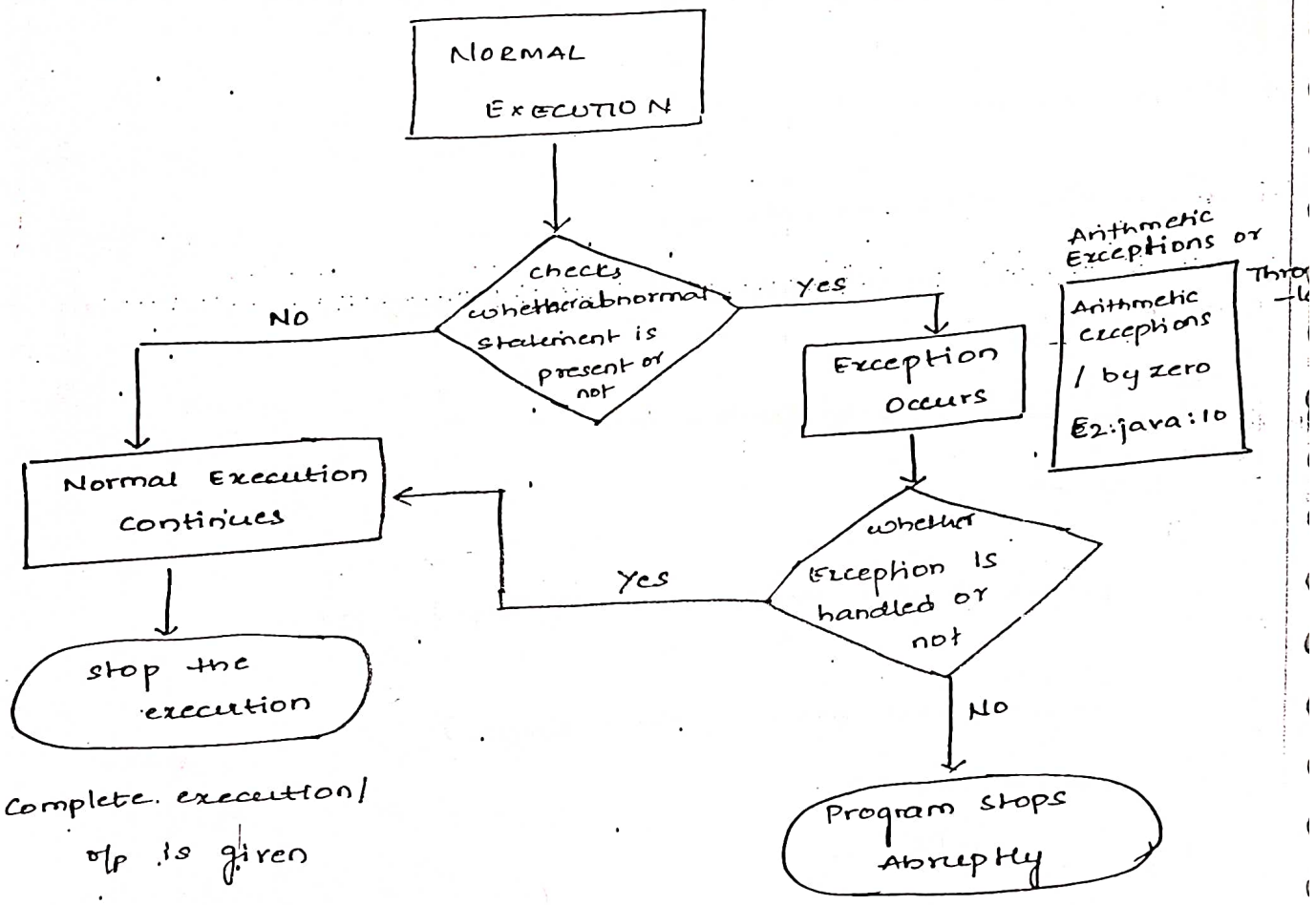
```
class E2
{
    public static void main (String[] args)
    {
    System.out.println ("Main Begin");
    int a=10;
    S.o.pln (1);
    int b= 0;
    S.o.pln (2);
    int ares = a/b; // abnormal statement
    S.o.pln ("Result :" + res);
    S.o.pln ("Main End");
    }
}
```

Opp:

```
Main Begin
1
2
ArithmeticException: / by zero at E2.java (E2.java:10)
```

· whenever the exception occurs a throwable type of object is created.

What happens when the Exception occurs ?



Every program written in java can undergo any one of the following execution flows.

1) Normal Execution flow

2) Exception execution flow.

▶ NORMAL EXECUTION FLOW:

The execution of a program starts normally, checks for every instruction whether it is an abnormal statement or not. In the normal execution we will not have any abnormal statement. Therefore the execution of the progra continues normally, executes all the instructions, gives us th complete o/p & then stops normally.

▶ EXCEPTION EXECUTION FLOW:

The execution of the program starts normaly, checks for the instruction whether it is abnormal statement or not. In the exception flow we will have an abnormal statement. As soon as it encounters an abnor statement immediately an exception object / a throwable type object will be created which will have the details like :

a) what is the name of the Exception.

b) why the Exception occurred.

c) where the Exception is occurring.

Once the exception object is created, the controller checks whether the exception is handled or not, if the exception is handled, the program goes for normal execution, gives us the complete o/p & stops normally.

If the exception is not handled then the pgm stops abruptly & gives us the partial output only till the abnormal statement ie the instructions present after the abnormal statement will never get executed.

▶ CLASSIFICATION OF EXCEPTION:

The exceptions are classified based on the compiler awareness. In java, we have 2 types of exception:

1. Checked exception.
2. Unchecked exception.

▶ CHECKED EXCEPTIONS:

The exceptions which are known to compilers are known as checked exceptions or the exceptions which are aware by the compilers are known as Checked Exceptions.

Whenever we have a checked exception, it is mandatory for the programmer to handle or declare the exception. Else we get compile time error (error: unreported exceptic ~~error~~ FileNotFound~~Exception~~)

Eg: FileNotFoundException, IOException, InterruptedException, SqlException, etc.

▶ UNCHECKED EXCEPTION:

The exceptions which are not known to the compiler are known as Unchecked Exceptions. or the compiler unaware exceptions are known as unchecked exceptions.

It is not mandatory to handle these exceptions.

Eg for checked Exception:

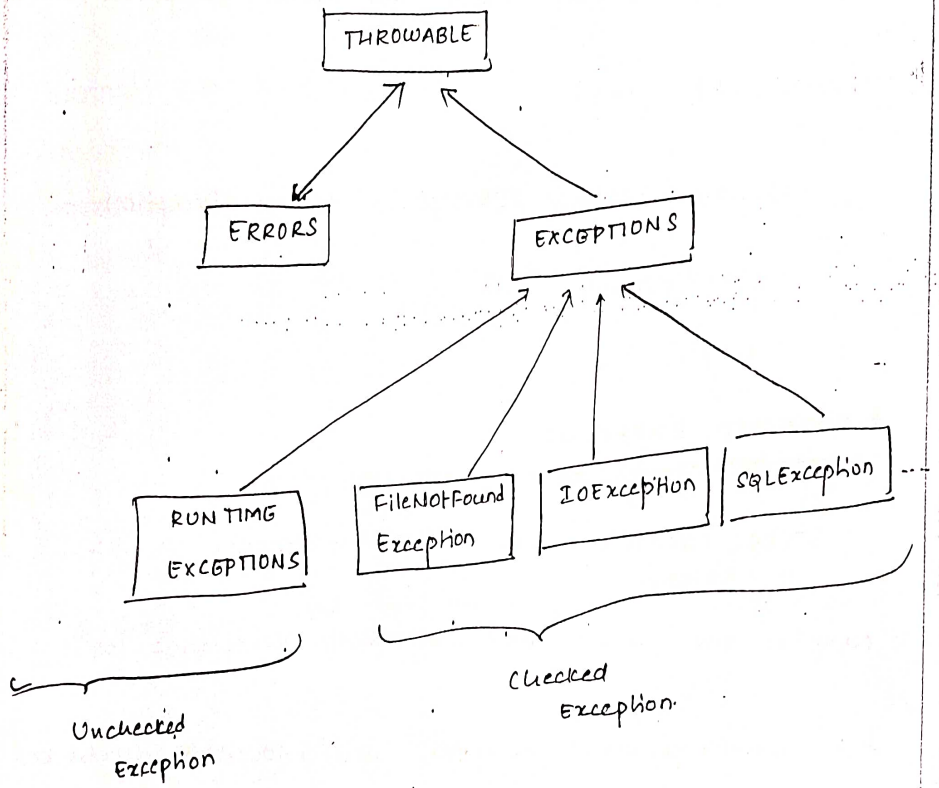
```
import java.io.FileOutputStream;

class Eg
{
    public static void main(String[] args)
    {
        String path = "D://java//filehandling//F1.txt";
        FileOutputStream fout = new FileOutputStream(path);
        System.out.println(fout);
    }
}
```

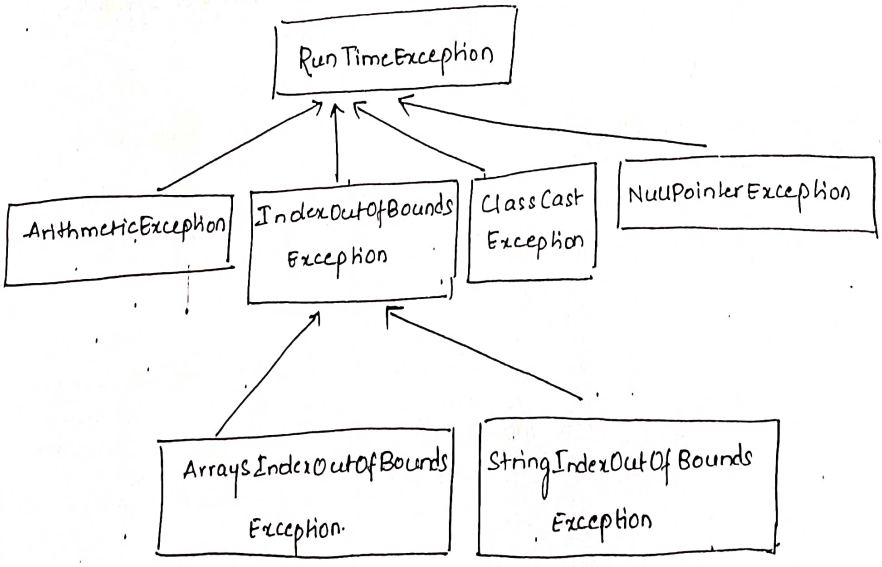
o/p: CTE

error: unreported exception : FileNotFoundException.

▶ HIERARCHY OF THROWABLE CLASS :



▶ HIERARCHY OF RUN TIME EXCEPTION :



EXCEPTION HANDLING :

We can handle the exception with the help of try and catch.

try and catch :-

SYNTAX :-

```
try {
    // abnormal statement;
}
catch (Throwable type reference variable) {
    // Java statements;
}
```

NOTE : Catch block must be declared immediately after try block

try block :

Inside try block we should always have the instructions responsible for exceptions.

When the exception occurs :

1) The execution of try block will be stopped.

2) A throwable type object is created. & if it is throc

3) to catch block.

▶ catch block:

1) Inside catch block we must always have a throwable type reference variable.

2) It is used to catch the reference of object thrown by try block.

3) We can say Exception is handled only when the object is thrown from the try is caught by the catch block else we say the exception is not handled.

NOTE: A catch block can catch only if the variable declared is of either of same type or the super class type of throwable heirarchy of object thrown.

Ex: class E4
```
{
  public static void main(String[] args)
  {
    System.out.println("Main Begin");
    try
    {
     int a=10;
     S.o.pln(1);
     int b = 0;
     S.o.pln(2);
     int res = a/b; //abnormal statement
     S.o.pln("Result : "+res);
```
}

```
    catch (RuntimeException e)
    {
        System.out.println("Caught....!");
    }
    System.out.println("Main End");
}
}
```

o/p:

```
Main begin
1
2
Caught..!
Main End
```

NOTE2: A catch block gets executed only the exception object is caught.

NOTE3: When we have more than one exception occurring within same program, we can handle it with the help of try with multiple catch blocks.

➤ try with multiple catch:

```
try {
    // abnormal statements;
}
catch (Throwable type reference variable) {
    // java statements;
}
catch (Throwable type reference variable) {
```

```
    // java statements;
}
catch (Throwable type ref variable) {
    // java statements;
}
```

* ⇒ In try with multiple catches any one of the catch block will be executed. ⇒ The catch blocks will be executed in top to bottom order.

⇒ whenever we are working with try with multiple catch, the catch blocks must be designed in such a way that the catch blocks with subclass reference should be present at the first place & catch blocks with the supermost reference must be present at the last.

CASE 1 :
```
catch (ArithmeticException e) {
}
catch (Exception e) {
}
catch (RuntimeException e) {
}
```
} Compile time error

CASE 2 :
```
catch (ArithmeticException e) {
}
catch (RuntimeException e) {
}
catch (Exception e) {
}
```
} Compile Time Success.

Eg:

```
class Division1
{
public static void main (String [ ] args)
{
System.out.println ("Welcome to Division software...!");
try
{
Scanner s = new Scanner (System.in);
System.out.println ("Enter the value for numerator");
int num = s.nextInt();
System.out.println (" Enter the value for Denominator");
int den = s.nextInt();
int res = num/den;
System.out.println ("The result is :" + res);
}
catch ( ArithmeticException e)
{
System.out.println ("sorry for inconvinience, zero is not all
}
catch(Exception e1)
{
System.out.println ("Enter numericals");
}
System.out.println ("Thank you for using the Application");
}
```

o/p:

```
Enter the value for numerator
10
Enter the value for denominator
0
Sorry for inconvinience, zero is not allowed
Thank you for using the Application
```

o/p 2:

```
Enter the value for numerator
20
Enter the value for denominator
10p
Enter incomericals
Thank you for using the application.
```

11/03/2022

▶ finally :

1) finally is a keyword

2) we can have finally block only in following situation

i) try block with finally block.

```
try {
    // abnormal statements;
}
finally {
    // java statements;
}
```

ii) finally with try & catch block.

```
try {
  // abnormal statements;
}
catch (Exception e)
{
  // javastatements;
}
finally
{
  // java statements;
}
```

iii) finally with try & multiple catch blocks.

```
try {
  // java abnormal statements;
}
catch (throwable type ref var)
{

}
catch (throwable type ref var)
{

}
finally {
   // java statements
}
```

FLOW OF EXECUTION FOR finally block:

1) If Exception not occurs, then also finally block gets execution.

2) If Exception occurs & even if it is handled, then also finally block gets execution.

3) If Exception occurs & not handled, then also finally gets executed.

EXCEPTION OBJECT PROPAGATION:

The movement of Exception Object from called method to calling method is known as Exception object propagation.

The Exception object will propagate only when the exception is not handled inside called method.

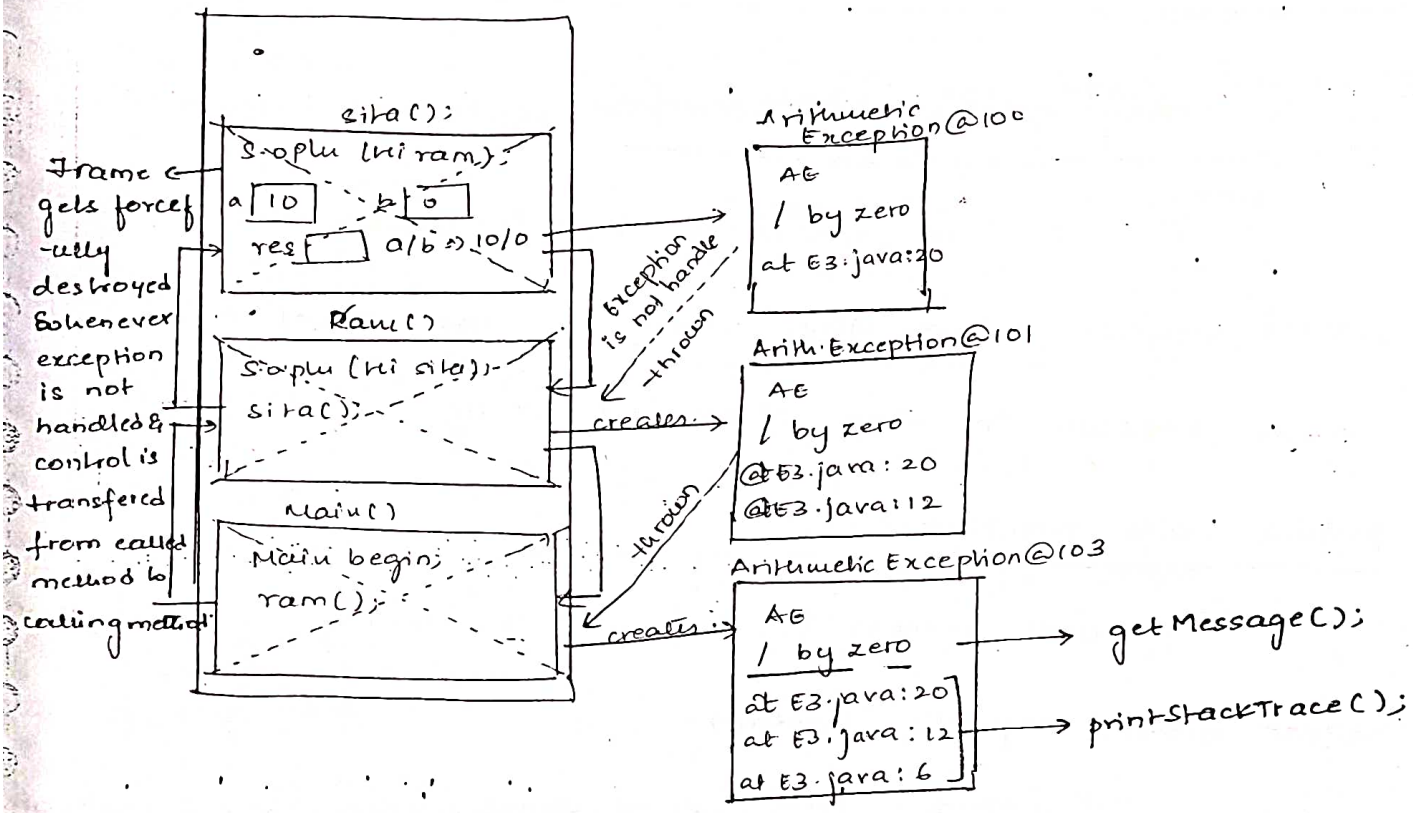
The exception object propagation is required in order to have the complete execution of the program.

Eg: class E3

```
{
 public static void main (String[] args)
 {
  System.out.println("Main Begin");
Ln.6:  ram();
  System.out.println("Main End");
 }
 public static void ram()
 {
  System.out.println("Hi sita");
Ln.12:  sita();
  System.out.println("Bye sita");
 }
 public static void sita()
 {
  System.out.println("Hi Ram");
  int a=10;
  int b=0;
Line no: 20:  int res=a/b; // Abnormal statement
  System.out.println("res");
  System.out.println("Bye Ram");
 }
}
```

O/P:

```
Main Begin
Hi sita
Hi Ram
Exception: Arithmetic Exception: / by zero
        at E3.java 20   at E3.java 12
```



NOTE: The Exception occured in the above program can be handled in 3 different places. they are.

(i) inside sita()

(ii) inside ram()

(iii) inside main(String [] args)

▶ THROWABLE CLASS :

* It is the supermost class of Exception hierarchy.
* Inside throwable class, toString() of object class is overriden in such a way that instead of printing the address of object, It will print fully Qualified name of the exception along with the reason.

▶ METHODS OF THROWABLE CLASS:

1) public String getMessage():

It is a non-static method present inside throwable class. which returns the name of the Exception along with reason. in the form of String

2) public void printStackTrace():

It will return us the stack tracing of exception object from top to bottam.

StackTrace is nothing but the flow of Exception object from Top to bottam order inside stack.

▶ throws:

It is a keyword which is used to declare the exception. It must be used along with declaration of method only.

We can throw multiple Exception using throws keywords.

If a checked Exception is not handled. using try and catch block, then it must be declared for exception object propag

-n.

SYNTAX:

```
[Modifier] returntype methodName([Formal Arg]) throws Except
                Exception2, Exception3 ..... Exception n;
```

NOTE: If we call a Method which declares the except using throws, then the calling method must either have it or throw it. Else we get compile time error.

```
import java.io.*;
Ex: class E4
{
 public static void main (String [] args) throws FileNotFoundEx
 {
  S.o.pln ("Main Begin");
  ram();
  S.o.pln ("Main End");
 }
 public static void ram() throws FileNotFoundException
 {
 S.o.pln ("Hi sita");
  sita();
 S.o.pln ("Bye sita");
 }
 public static void main sita () throws FileNotFoundException.
 {
 S.o.pln ("Hi Ram");
 FileNotFoundE
 FileOutputStream fout = new FileOutputStream ("C:// Users/demo/B/
 S.o.pln (fout);
 S.o.pln ("Bye Ram");
 }
}
```

O/P:

Main Begin

Hi Sita

Hi Ram

~~File Not~~ java.io.FileOutputStream @ some hexadecimal-value

Bye Ram

Bye Sita

Main End

Can a user create his own Exception ?

Yes, user can create his own exception with the help of throw keyword.

▶ **throw :**

It is a keyword.

* It is used to create the user defined exception & stop the flow of execution.

SYNTAX:

```
throw new ClassName();
```

Eg: throw new SalaryException();

▶ CUSTOM EXCEPTION OR USER DEFINED EXCEPTION:

The exception created by the user are known as Custom Exception / User defined Exception.

The user can create his own exception by following below steps.

1) Create a new class of Exception.

2) Extend the created class to any one of the throwable cl

NOTE: If newly created class Inherit any of the unchecd exception class, then it behaves like an unchecked excep

If newly created class Inherit any of the checked excep classes, then it behaves like checked exception.

Eg: class ~~Employe~~ SalaryException extends RuntimeException

```
{
}
class Employee
{
int eid;
String name;
double sal;
Employee () { }
Employee (int eid, String name, double sal)
 {
  try
  {
   this.eid = eid;
   this.name = name;
   if (sal < 0)
    {
     throw new SalaryException ();
    }
```

```
        else
          {
          this.sal=sal;
          }
      }
      catch (Exception e)
      {
        System.out.println("Salary can't be lesser than zero");
      }
    }
    public String toString()
    {
      return "name;"+name+";sal:"+sal;
    }
  }
  class Driver
  {
    public static void main(String[] args)
    {
      Employee e1 = new Employee(01, "Manasa", -10000);
      System.out.println(e1.sal);
    }
  }
```

<u>O/P:</u>

Salary can't be lesser than zero

0.0

throws	throw
* It is used to declare the exception & not to handle	* It is used to create the exception & stop the normal flow of execution
* It can only be used along with method declaration statement	* It can be used inside the block.